

Step 3 – Design Deep Dive

In the high-level design, we briefly discussed two major flows:

1. Feed Publishing
2. News Feed Building

In this section, we dive deeper into the internal working of these components.

Feed Publishing Deep Dive

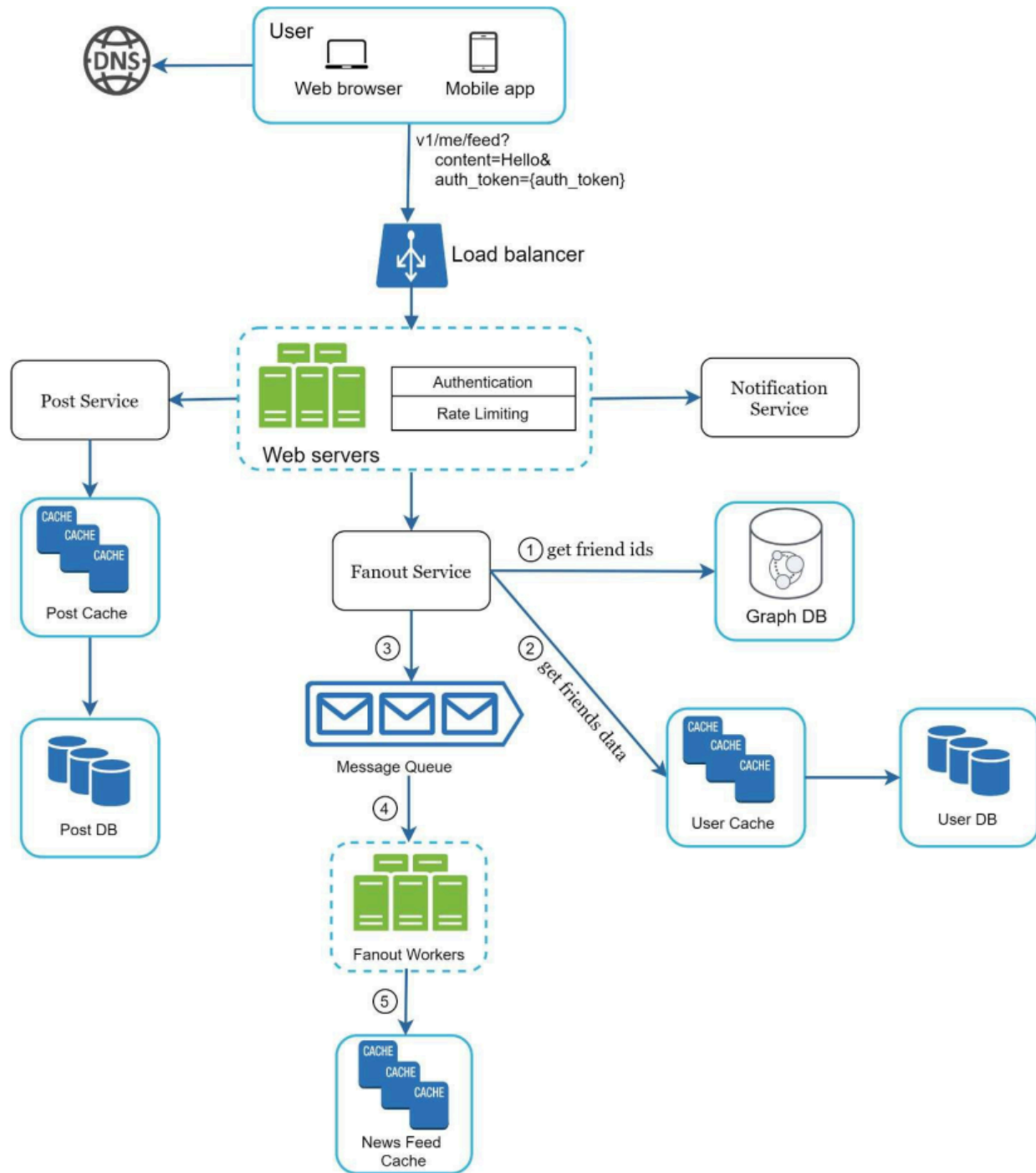


Figure presents the detailed architecture of the feed publishing flow.

The main focus areas are:

- Web Servers

- Fanout Service

Feed Publishing Workflow

When a user publishes a post:

1. Request reaches web servers
2. User authentication is verified
3. Post is stored
4. Fanout service distributes the post
5. Feed cache is updated
6. Friends can immediately see the post

Web Servers

Web servers act as the entry point of the system.

Besides communicating with clients, web servers also handle critical security and protection mechanisms.

Responsibilities of Web Servers

Responsibility	Description
Authentication	Verify logged-in users
Rate Limiting	Prevent spam and abuse

Routing	Forward requests to internal services
Validation	Validate post requests

Authentication

Only authenticated users are allowed to publish posts.

Authentication Mechanism

The system verifies:

None

`auth_token`

before accepting requests.

Purpose

Authentication ensures:

- Only valid users can create posts
- Unauthorized requests are blocked
- System security is maintained

Rate Limiting

Rate limiting is enforced to prevent:

- Spam
- Abuse
- Excessive posting

Example

The system may restrict:

None

Maximum number of posts per user within a fixed time window.

Benefits of Rate Limiting

- Prevent server overload
- Reduce malicious activity
- Improve system reliability

Fanout Service

The fanout service is one of the most important components in a news feed system.

What is Fanout?

Fanout is the process of:

None

Delivering a new post to followers or friends.

Two Types of Fanout Models

There are two major fanout approaches:

Model	Alternative Name
Fanout on Write	Push Model
Fanout on Read	Pull Model

1. Fanout on Write (Push Model)

In this approach:

None

Feed is precomputed during write time.

When a user creates a post:

- The post is immediately pushed to friends' feeds
- Feed cache is updated instantly

Workflow

1. User publishes a post
2. System fetches friend list
3. Post is inserted into each friend's feed cache

Advantages of Push Model

1. Real-Time Feed Updates

Friends receive updates immediately.

2. Fast Feed Retrieval

Feed is already precomputed.

Thus:

None

Read latency becomes very low.

Disadvantages of Push Model

1. Hotkey Problem

If a celebrity has millions of followers:

- Fanout becomes expensive
- System overload may happen

This is called:

Hotkey Problem

2. Wasted Computation

Inactive users may never open the feed.

Still, the system computes feeds for them.

This wastes:

- CPU
- Memory
- Network resources

2. Fanout on Read (Pull Model)

In this model:

None

Feed is generated only when the user requests it.

No precomputation happens.

Workflow

1. User opens news feed
2. System pulls recent posts dynamically
3. Feed is generated on-demand

Advantages of Pull Model

1. Better for Inactive Users

No unnecessary feed generation.

2. No Hotkey Problem

Posts are not pushed to all followers immediately.

This reduces:

- Write amplification
- System overload

Disadvantages of Pull Model

Slower Feed Retrieval

Feed generation happens during read time.

Thus:

None

Read latency becomes higher.

Hybrid Approach

Modern large-scale systems use:

Hybrid Fanout Model

to combine benefits of both approaches.

Strategy Used

User Type	Model Used
Normal Users	Push Model
Celebrities	Pull Model

Why Hybrid Approach?

It provides:

- Fast feed retrieval
- Better scalability
- Reduced hotkey issues
- Efficient resource usage

Consistent Hashing

The system may use:

Consistent Hashing

to distribute requests evenly across servers.

Benefits

- Reduces hotkey problem
- Improves load balancing
- Supports horizontal scaling

Detailed Fanout Service Workflow

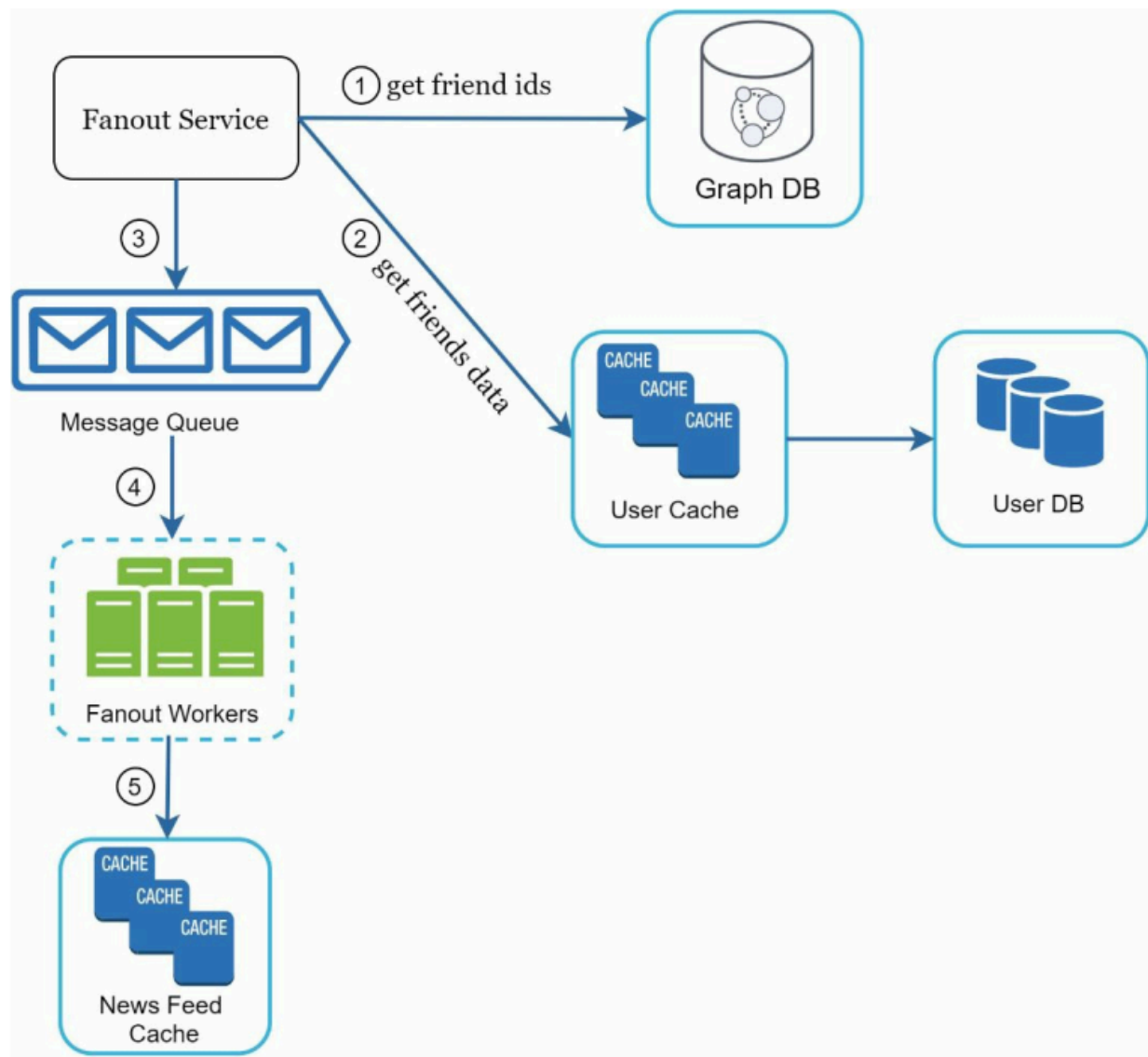


Figure explains the internal workflow of the fanout service.

Step 1 – Fetch Friend IDs

The system first retrieves friend relationships.

Graph Database

Friend relationships are stored in:

Graph Database

Why Graph Database?

Graph databases are ideal for:

- Friend relationships
- Social connections
- Friend recommendations

Step 2 – Fetch Friend Information

The system retrieves friend information from:

User Cache

Filtering Logic

The system filters users based on:

- Privacy settings
- Muted users
- Blocked users
- Selective sharing rules

Example

If User A muted User B:

None

Posts from User B will not appear in User A's feed.

Step 3 – Send Data to Message Queue

The system sends:

- Friend list
- Post ID

to:

Message Queue

Why Message Queues?

Message queues help:

- Decouple services
- Improve scalability
- Enable asynchronous processing

Step 4 – Fanout Workers Process Queue

Fanout workers:

- Consume queue messages
- Update feed cache

News Feed Cache Structure

The cache stores:

None

`<post_id, user_id>`

mapping.

Why Store Only IDs?

Storing full post objects in cache is expensive.

Thus, only IDs are stored to reduce:

- Memory usage
- Cache size

Feed Cache Optimization

Most users only read recent posts.

Therefore:

- Cache stores latest feed entries
- Older entries can be evicted

This keeps:

- Cache small
- Cache hit ratio high

Step 5 – Store Feed Data in Cache

post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id
post_id	user_id

The final feed mapping is stored in:

News Feed Cache

Example Cache Entry

user_id	post_id
101	P5001
101	P5002
102	P5001

Benefits of Feed Cache

- Faster feed retrieval
- Reduced database queries
- Better scalability
- Improved performance

Overall Deep Dive Summary

Component	Responsibility
Web Server	Authentication and rate limiting
Fanout Service	Distribute posts

Graph Database	Store friend relationships
User Cache	Fetch user information
Message Queue	Async processing
Fanout Workers	Update feed cache
Feed Cache	Store feed mappings

Key Takeaway

The most critical design decision in large-scale news feed systems is:

None

Choosing the right fanout strategy to balance write cost, read performance, and scalability.